

Quiz 6 Solution

Q1. Which one of the following statements is TRUE about the code given below?

```
[&,=foo](int a, int b) -> int {return foo(a,b); };
```

- a) Captures all by reference, but foo by value
- b) Captures all by value, but foo by reference
- c) Captures all by value
- d) None of the others

Correct: d), passing by value doesn't require mentioning the = operator explicitly for one variable

Q2: Which of the following best highlights a key difference between a Lambda function and a functor in C++?

- a) A capture list is mandatory for a lambda function, while it is not for a functor
- b) A lambda function can be overloaded, while a functor cannot
- c) A functor is an object with an overloaded operator(), while a lambda function is an inline compiler-generated unnamed function object
- d) A lambda function must be defined in a class, while a functor can be defined globally

Correct: c)

Q3. Why are Lambda functions also called anonymous functions?

- a) Because they use the auto keyword and are not tied to a specific class
- b) Because they do not have a declared return type by default
- c) Because they do not have an explicit name in the source code
- d) Because they are used for short operations inside functions

Correct: c)

Q4. Which of the following statements BEST describes functors in C++?

- a) Functors are objects that can act as functions
- b) Functors can act as functions that can also hold some local state
- c) Functors can be assigned to functional pointers
- d) All of the others

Correct: d)

Q5. Which of the following best describes the function signature given below?

```
double (*ptr)(void (*a)(string c),int (*b)(double d))
```

- a) A function that accepts two function pointers as parameters and returns a double, the first parameter function returns void and accepts a string, and the second returns an int and accepts a double
- b) A function that accepts two function pointers as parameters and returns a double, the first parameter function returns void and accepts a string, and the second returns an int and accepts an int
- c) A function that accepts two function pointers as parameters and returns a double, the first parameter function returns void and accepts a char, and the second returns an int and accepts a double
- d) None of the others

Correct: a)

Q6. The code below gives a compilation error. Which of the following best explains why?

```
#include <iostream>
using namespace std;

class Object {
public:
    int operator()(int a) { return a + 1; }
};

int main() {
    Object c;
    int (*fp)(int) = c; // ❌ Compilation error here
    cout << fp(5);
    return 0;
}
```

- a) The operator() must be declared as static for the object to be used like a function pointer
- b) You must use this->operator() when assigning a functor to a function pointer
- c) Function pointers expect actual function addresses. Object is a functor – a callable object – so it can't be directly assigned to a function pointer. To store and call a functor like a function, we need to use std::function<int(int)> fp = c;
- d) The return type of operator() is incompatible with function pointers

Correct: c)

Q7. What will be the output of the following code?

```
vector<int> nums = {1,2,3,4,5,6,7,8,9,10};  
  
vector<int> doubles;  
  
auto doubleAndStore = [](int a){ doubles.push_back(a*2); };  
  
for_each(nums.begin(),nums.end(),doubleAndStore);  
  
for_each(doubles.begin(),doubles.end(),[](int a){ cout<<a; });
```

- a) {1,2,3,4,5,6,7,8,9,10}
- b) {2,4,6,8,10,12,14,16,18,20}
- c) {}
- d) Compilation error

Correct: d)

Q8. Lambda Functions have superiority over Functors for which of the following reason(s)?

- a) Lambdas can capture variables by reference or by value
- b) Lambdas can be passed as arguments to functions that expect function pointers
- c) Lambdas can be used to define small, one-off functions inline without needing to create a separate function object
- d) All of the others

Correct: d)

Q9. What will be the output of the following code?

```
int x = 5;  
  
auto addFive = [x = 10](int a){ return a + x; };  
  
cout<<addFive(5);
```

- a) 10
- b) 15
- c) Compilation error
- d) None of the others

Correct: b)

Q10. Suppose the "this" pointer is captured by value in the capture list of a lambda function inside a class. Which of the following statements is correct?

- a) Any changes made to member variables inside the lambda function will be reflected outside the lambda
- b) Any changes inside the lambda function affect only a copy of the object and are not retained
- c) The "this" pointer cannot be used inside lambda functions defined in class methods
- d) None of the others

Correct: a)

Q11. Which of these correctly declares a function pointer named `fp` that points to a function taking two `int` parameters and returning a `bool`?

- a) `bool *fp(int, int);`
- b) `bool (*fp)(int, int);`
- c) `bool fp(*int, *int);`
- d) `(*fp)(int, int) bool;`

Correct: b)

Q12. Given these definitions:

```
int square(int x) { return x * x; }
```

```
int apply(int (*fn)(int), int value);
```

Which call correctly invokes `apply` with `square`?

- a) `apply(square, 5);`
- b) `apply(square(), 5);`
- c) `apply(int (*)(int) square, 5);`
- d) `apply("square", 5);`

Correct: a)

Q13. Given

```
std::vector<int> in = {1, 2, 3};  
std::vector<int> out(in.size());
```

Which call correctly uses a lambda function to double each element from `in` into `out`?

- a) `std::transform(in.begin(), in.end(), out.begin(), [](int x){ return x * 2; });`
- b) `std::transform(out.begin(), out.end(), in.begin(), [](int x){ return x * 2; });`
- c) `std::copy_if(in.begin(), in.end(), [&](int x){ out.push_back(x * 2); });`
- d) `std::transform(in.begin(), in.end(), std::back_inserter(out), [](int x){ x * 2; });`

Correct: a)

Q14. Given this functor and code:

```
template<typename T>  
class Filter {  
    T limit;  
public:  
    Filter(const T& t): limit(t) {}  
    void operator()(const T& value) const {  
        if (value < limit) std::cout << value << ' ';  
    }  
};
```

```
std::vector<int> v = {2, 5, 1};  
std::for_each(v.begin(), v.end(), Filter<int>(3));  
// for each does something for each element of array as implied by the name  
What is the output?
```

- a) 2 5 1
- b) 5 1 2
- c) 2 1
- d) No output

Correct: c)

Q15. Why are functors (function objects) more flexible than plain function pointers?

- a) They can be passed to `std::sort`
- b) They allow the overloading `operator()`
- c) They can maintain internal state between calls
- d) They guarantee inlining by the compiler

Correct: c)

Q16. Which lambda capture clause makes all local variables available by reference but captures `threshold` by value?

- a) `[&, threshold]`
- b) `[=, threshold]`
- c) `[threshold, this]`
- d) `[threshold]`

Correct: a)

Q17. What is the type of a C++ lambda expression?

- a) None of the others
- b) An unnamed type that overloads the `operator()`
- c) A predefined object type from `<functional>`
- d) A built-in functor type in the STL

Correct: b)

Q18. Which of these is a syntactically valid lambda function that takes two `int`'s and returns their sum?

- a) `[] (int a, int b) { a + b; }`
- b) `[] (int a, int b) { return a + b; }`
- c) `[] (int a, int b) -> void { return a + b; }`
- d) `[] (int a, int b) -> auto { a + b; }`

Correct: b)

Q19. To "reduce" a sequence of `int` values into a single sum in C++, which STL algorithm do you use?

- a) `std::transform`
- b) `std::accumulate`
- c) `std::copy_if`
- d) `std::remove_if`

Correct: b)

```
Q20. int main(){  
    int a=5,c=6;  
    auto b = [](int x){return x*a;}  
    int result = b(c);  
    cout << result;  
    return 0;  
}
```

What will be the output?

- a) 30
- b) true
- c) 6
- d) Compiler Error

Correct: d)